Knowledge Graph Pipeline

Michael Jackson Knowledge Base

Web Mining & Semantics — Final Project Report

Laura Aboukrat & Laura Deleuze — 2026

Abstract

This report documents a complete Knowledge Graph pipeline that we built around the domain of Michael Jackson. Starting from raw web pages, we explored 10 sources, extracted 2,005 entities, and built an RDF knowledge base which grew from 5,202 triples to 66,719 triples after alignment with Wikidata and SPARQL expansion. We then applied SWRL reasoning rules, trained four Knowledge Graph Embedding models — with the RotatE model achieving our best result with an MRR of 0.196 — and finally built a RAG chatbot powered by a local Gemma 2B model which correctly answered 3 out of 5 questions during testing.

1. Data Acquisition & Information Extraction

The first step of the project was to gather raw data from the web and extract structured information from it. This phase sets the quality bar for everything that follows: the better the entities we extract here, the better our Knowledge Base will be down the line.

1.1 Choosing the Domain and Seed URLs

We chose Michael Jackson as our domain because he is one of the most extensively documented public figures in music history. His career spans several decades, involves a large number of collaborators, awards, albums and cultural events, which means there is a rich variety of entities and relations to extract. It also helps that Wikidata has a very detailed entry for him (wd:Q2831), which made the alignment step much easier. We selected 10 seed URLs covering different types of sources: the French Wikipedia article, Larousse and Universalis encyclopedia entries, music-focused websites like NRJ, and a few fan sites and archives. The goal was to get diverse coverage rather than scraping the same content multiple times.

1.2 Crawling and Cleaning

For the crawling pipeline, we used the trafilatura library, which is specifically designed to extract the main textual content from a webpage while removing all the noise: navigation menus, cookie banners, ads, footers, and so on. This was a deliberate choice over BeautifulSoup, because with BeautifulSoup we would have had to write custom rules for each website, while trafilatura handles this automatically. We added a quality filter: any page with fewer than 500 words after extraction was discarded as too sparse to be useful. All results were stored in a JSONL file (one JSON object per page), preserving the mapping between each URL and its extracted text. In the end, all 10 pages passed the filter, with the Wikipedia article alone yielding over 24000 words.

1.3 Named Entity Recognition

We ran spaCy NER on all 10 crawled documents. One decision we had to make here was the choice of model. The lab instructions suggest en_core_web_trf (English transformer model), but since most of our sources were in French, we used fr_core_news_lg instead. This was a trade-off: the French model gives better results on French text, but it also means that some entities come out in French

forms that are harder to align with Wikidata later. After cleaning and deduplication, we extracted 2005 entities across four types:

Type	Count	What it captures
PERSON	726	Individuals: Michael Jackson, Quincy Jones, Elvis Presley, Berry Gordy...
ORG	268	Organizations and labels: Motown, Jackson Five, Epic Records, ABC...
GPE	273	Locations: Los Angeles, Indiana, Gary, Paris...
MISC	738	Other named items: Grammy, Thriller, Billboard, MTV...

These 2005 entities were saved to entities_mJackson.csv with their type and source URL, and fed into the next step for Knowledge Base construction.

1.4 Relation Extraction via Dependency Parsing

Beyond just extracting entities, we also used spaCy's dependency parser to find candidate relations between entities co-occurring in the same sentence. The idea is to look for a verb that connects two entities, using the dependency tags nsubj (nominal subject) and dobj (direct object):

nsubj → ROOT verb ← dobj.

For example, from the sentence 'Michael Jackson signed a contract with Epic Records', the parser identifies Michael Jackson as nsubj, signed as the ROOT verb, and Epic Records as part of the object. This gives us the candidate triple (Michael_Jackson, signer, Epic_Records), which later gets aligned to wdt:P264 (record label) in our ontology.

This dependency-based extraction is imperfec. it misses many relations and sometimes extracts wrong ones — but it gave us a useful starting point for defining our private predicates before the Wikidata alignment step.

1.5 Ambiguity Analysis

Working with NER always involves dealing with ambiguous cases. For example we have 'Jackson which' was extracted as a PERSON entity. But it is ambiguous between Michael Jackson himself and references to the Jackson family as a group. The model cannot distinguish these without broader context. We kept both instances but flagged duplicates during KB construction. Also we have 'Billie Jean' which was classified as PERSON because of its first-name and surname structure, even though it is a song title. This is a typical case where domain knowledge is missing from the general-purpose NER model. We manually filtered out known song titles from the PERSON entity list. Also 'Grammy Awards47' appeared as a single entity because the HTML source had a number concatenated directly to the award name with no space. This was an artifact of the scraping, not a real entity. We applied a regex post-processing step to strip trailing numbers from entity labels.

1.6 Scaling Reflection

If we had to scale this pipeline from 10 pages to 10000, things would need to change significantly. Sequential crawling with requests would be far too slow ; we would need a distributed crawler like Scrapy with parallel workers. spaCy's nlp.pipe() allows batch processing which is much faster than processing documents one by one. We would also need to move from JSONL files to a proper database (PostgreSQL or Elasticsearch) to handle the volume and enable indexed search. Finally, at that scale, deduplication becomes critica. We have many pages repeat the same information, and without near-duplicate detection. We would end up with a very noisy KB.

2. Knowledge Base Construction, Alignment & Expansion

The goal of this phase was to transform our raw entity list into a proper RDF Knowledge Base, connect it to Wikidata. And after expand it to support Knowledge Graph Embedding training.

2.1 Building the Initial Private KB

We modeled the initial KB in RDF using a custom namespace http://mjackson.org/resource/ for all entities. First, we used URIs rather than plain strings for all entities, which is a requirement for proper RDF modeling. Second, we named predicates in camelCase (enregistrer, recevoir, isRelatedTo) following standard ontology conventions. Third, we made sure to remove duplicate triples before serializing the graph.

The resulting initial KB contains 5162 triples, 1048 unique entities and 39 distinct predicates which are above the minimum requirements of 100 triples and 50 entities.

2.2 Entity Linking with Wikidata

For each of the 1052 unique entities, we queried the Wikidata search API to find the best matching Wikidata entry. The API returns a confidence score that we stored in our alignment table (entity_alignment.xls). For well-known entities, this worked very well: Janet Jackson (Q13553631, score=1.0), John Lennon (Q1203, score=1.0), and CBS (Q43380, score=1.0) were all matched correctly.

However, many entities extracted from French sources failed to align. Entities like 'Epic_Records30', 'Grammy_Awards47' or 'Biographie_courte' got a score of 0.0 because the Wikidata API could not match these French-form or malformed labels. This is a consequence of our choice to use a French NER model.

For aligned entities, we added owl:sameAs triples linking our private URIs to the corresponding Wikidata URIs. For entities that could not be found in Wikidata, we created new semantic definitions using rdfs:subClassOf and rdfs:domain/range axioms in our ontology file.

2.3 Predicate Alignment

We aligned our private predicates to Wikidata properties using two strategies. The first was direct matching: given a pair of aligned entities (e.g., MJ and Epic Records), we queried Wikidata to find which properties directly connect them. The second was textual matching, using FILTER(CONTAINS()) on property labels to find semantically similar properties.

The predicate alignments are stored in ontology_alignment.ttl. The most important ones are:

Private Predicate	Wikidata Property	Relation Type	Meaning
prop:recevoir	wdt:P166	owl:equivalentProperty	award received
prop:signer	wdt:P264	owl:equivalentProperty	record label
prop:rejoindre	wdt:P463	owl:equivalentProperty	member of
prop:enregistrer	wdt:P358	owl:equivalentProperty	discography
wdt:P737	prop:isRelatedTo	rdfs:subPropertyOf	influenced by

One difficulty we ran into was that the direct SPARQL queries for key pairs (e.g., MJ ↔ Epic Records, MJ ↔ Diana Ross) returned empty results. After investigation, we found this was because the query structure required exact direct property paths between the two entities, but Wikidata sometimes

models these relationships through intermediate nodes. We resolved this by switching to textual property search, which was more robust.

2.4 KB Expansion Strategy

To reach the volume needed for Knowledge Graph Embedding (50k-200k triples), we expanded the KB using SPARQL queries on the Wikidata endpoint. We used three levels of expansion, each building on the previous one.

1-hop expansion

We started by collecting all direct properties of wd:Q2831 (Michael Jackson): his music genres, awards, record labels, collaborators, family members, places of birth and death, and so on. This gave us a first circle of entities directly connected to MJ. `SELECT ?p ?o WHERE { wd:Q2831 ?p ?o . } LIMIT 1000`

Predicate-controlled expansion

We then expanded along specific predicates that are semantically central to our domain. For example, collecting all entities connected via P166 (award received) gives us not just MJ's awards, but all other artists who received those same awards — which creates useful connections in the graph. `SELECT ?s ?p ?o WHERE { ?s wdt:P166 ?o . } LIMIT 20000`

2-hop expansion

Finally, we collected properties of the entities already in our graph. For instance, once we had MJ's awards as nodes, we expanded to include the properties of those award entities. This enriched the graph structure. `SELECT ?award ?p ?o WHERE { wd:Q2831 wdt:P166 ?award . ?award ?p ?o . } LIMIT 10000`

The final expanded KB statistics are:

Metric	Value	Requirement	Status
Total triples	66,719	50,000 – 200,000	✓ OK
Unique entities	54,768	5,000 – 30,000	Slightly above
Unique relations	98	50 – 200	✓ OK

The entity count is slightly above the recommended maximum of 30,000. This happened because the 2-hop expansion pulled in many peripheral entities that are only weakly connected to the core MJ subgraph. In hindsight, we should have applied stricter predicate filtering to keep the graph more focused.

3. Knowledge Reasoning with SWRL

This section covers the reasoning part of the project. We apply rule-based inference to derive new knowledge from existing facts. After that we compare this approach with what embedding models can capture.

3.1 SWRL Rule on family.owl — oldPerson

We loaded the family.owl ontology (12 individuals, 16 classes, 13 properties) using OWLReady2 and defined the following SWRL rule: `Person(?p) ∧ age(?p, ?a) ∧ greaterThan(?a, 60) → oldPerson(?p)`

This rule says that any person with an age property greater than 60 should be classified as an oldPerson. Since Pellet and HermiT reasoners require Java and were not available in our

environment, we implemented the reasoning manually in Python by iterating over all individuals and checking their age property. The rule correctly classified Peter (age 70) and Marie (age 69) as oldPerson instances, while the younger individuals were left unclassified.

Individual	Age	Classified as oldPerson?
Peter	70	✓ Yes
Marie	69	✓ Yes
John	45	No
Thomas	40	No
Paul	38	No
Sylvie	30	No

3.2 SWRL Rule on the Michael Jackson KB

For our KB, we designed a Horn rule with two conditions inspired by the idea that artistic influence often comes with shared recognition: P737(p, a) ∧ P166(a, award) → P166(p, award)

This means that if person p is influenced by artist a (P737 = influenced by), and a received a given award (P166 = award received), then we infer that p likely received the same award too. This is a simplification of being influenced by someone doesn't guarantee you win the same awards but as a rule for knowledge inference it is reasonable enough to be interesting.

The rule inferred 3,812 triples in total, of which 3,785 were genuinely new facts not already in the KB. Only 27 triples were already known. This shows that even a simple two-condition rule can generate a significant amount of new knowledge when applied to a large graph.

Person (p)	Influenced by (a)	Inferred Award	Already in KB?
Q39803	Q1325	Q17144	No ✓
Q76519	Q937	Q28003	No ✓
Q4235	Q303 (MJ)	Q17985761	No ✓

3.3 Comparing Rule-Based and Embedding-Based Reasoning

To check whether our RotatE embedding model captured the same pattern as our rule, we tested vector composition: vector(P737) * vector(P166) ≈ vector(P166) ?

The result was a cosine similarity of only 0.017 between the composed vector and P166, which means the embedding did not confirm the rule. The closest relations found were P1303 (instrument, 0.248), P106 (occupation, 0.210) and P3373 (sibling, 0.183). none of which are semantically related to our rule. We were not completely surprised by this result. Given the facts in the KB, the rule produces logically guaranteed conclusions. Embedding-based reasoning is inductive, the model learns statistical patterns from data, and if the training data is noisy or the model has not converged, those patterns may not align with specific logical rules. In our case, 100 epochs on CPU is likely not enough for RotatE to fully capture compositional patterns. The open-world assumption of RDF also

means many valid triples are simply absent from the KB, which confuses the embedding model during training.

So the two approaches are complementary rather than competing. We have rules which are great for precise, domain-specific inference and embeddings which are great for generalization and handling incomplete data.

4. Knowledge Graph Embeddings

The goal of this phase was to learn vector representations of entities and relations in our KB, enabling link prediction and semantic analysis. We used the PyKEEN framework and trained four different models to compare their performance.

4.1 Data Preparation

Before training, we cleaned the expanded KB. We removed duplicate triples, normalized entity URIs, and ensured consistent indexing of all entities and relations.

We applied an 80/10/10 train/validation/test split, with one important constraint: no entity can appear only in the validation or test set. This constraint prevents the model from being evaluated on entities it has never seen during training. Because our Wikidata-derived graph contains many entities that appear in only one or two triples, this filtering reduced the effective validation and test sizes significantly:

Split	Triples	Effective %
Train	53,375	94.1%
Validation	1,637	2.9%
Test	1,699	3.0%

The split ends up looking like 94/3/3 rather than 80/10/10. This is expected and correct. the entity containment constraint is mandatory to avoid data leakage, and it is a consequence of working with sparse knowledge graphs. The 20% of triples initially intended for validation/test contained too many rare entities to be kept.

4.2 Model Descriptions and Training Configuration

We trained four models with consistent hyperparameters to ensure a fair comparison: embedding dimension 100, learning rate 0.01, batch size 512, 100 epochs, on CPU. We have :

TransE represents relations as translation vectors: we get a head entity h and a relation r, the model expects h + r ≈ t (the tail entity). It is simple and fast to train, but it struggles with one-to-many relations (h + r cannot equal multiple different t values simultaneously)

DistMult uses element-wise product of entity and relation vectors. It handles symmetric relations well but cannot model antisymmetric ones. This is a problem for directional relations.

ComplEx extends DistMult to the complex number domain. This allows to model asymmetric relations. However, it requires longer training to converge compared to DistMult.

RotatE represents relations as rotations in complex space. This formulation can handle symmetric, antisymmetric, inverse, and compositional relations. This is the best one

4.3 Results and Interpretation

Model	MRR	Hits@1	Hits@3	Hits@10	MRR Head	MRR Tail
TransE	0.0465	0.0128	0.0360	0.1115	0.0431	0.0499
DistMult	0.1299	0.0687	0.1393	0.2489	0.0920	0.1678
ComplEx	0.0479	0.0241	0.0507	0.0925	0.0341	0.0617
RotatE	0.1958	0.1426	0.2126	0.3012	0.1506	0.2411

RotatE is clearly the best model with MRR=0.196 and Hits@10=0.301. This matches our expectations. Wikidata contains many asymmetric and directional relations which RotatE is specifically designed to handle via rotation in complex space.

DistMult comes second with MRR=0.130. Despite its theoretical inability to model antisymmetric relations, it performs reasonably well because many Wikidata triples do not require strict asymmetry to score correctly.

ComplEx's poor performance (MRR=0.048, worse than DistMult) was actually surprising to us. Theoretically, ComplEx should outperform DistMult on asymmetric relations. We think that 100 training epochs on CPU is insufficient for the complex-valued optimization to converge properly, while DistMult's simpler formulation converges faster.

TransE is the weakest model (MRR=0.047), which is entirely expected. In a Wikidata graph, many relations are one-to-many: for example, wd:Q2831 (MJ) has 22 different awards via wdt:P166. TransE's assumption that h + r maps to a single point in embedding space fails completely in this scenario.

We notice that MRR_tail > MRR_head for all models. This means predicting the tail entity given (head, relation) is easier than predicting the head entity given (relation, tail). In a knowledge graph centered on a famous person, this makes sense: many triples have MJ as the subject (head), and his connections are very diverse, making head prediction harder.

4.4 KB Size Sensitivity

We trained RotatE on three subsets of our KB to observe how performance scales with data volume:

KB Size	Train Triples	Entities	RotatE MRR	RotatE Hits@10
20k	16,000	15,198	0.0719	0.1303
50k	40,000	34,535	0.1424	0.2404
Full (66k)	53,375	44,788	0.1745	0.2869

The results confirm what the theory predicts. more we have data more the performance impooves. We pass from 20k to the full dataset, ans we see that MRR nearly triples (0.072 to 0.175) and Hits@10 more than doubles (0.130 to 0.287). The 20k subset performs particularly poorly, with an MRR of only 0.072, which is consistent because small KBs produce unstable embeddings. At 20k triples, the graph is too sparse for the model to learn meaningful structural patterns.

4.5 Embedding Analysis

Nearest Neighbors

We looked at the nearest neighbors of Michael Jackson (Q2831) in the RotatE embedding space, measured by cosine similarity:

Rank	Entity QID	Cosine Similarity
1	Q317784	0.6727
2	Q131324	0.6570
3	Q26695	0.5760
4	Q311238	0.5690
5	Q131814	0.5638

The cosine similarities are relatively high (above 0.55), suggesting the model placed semantically related entities close to MJ in the embedding space. The entities are identified by Q-ids only in our graph (we did not include rdfs:label triples in the KB). so resolving their exact names would require an external Wikidata lookup. Based on the structure of the graph, these are most likely musicians or entertainers who share similar award, genre, and label connections with MJ.

t-SNE Visualization

We applied t-SNE to 2000 entity embeddings (we only use the real part of RotatE's complex vectors, because t-SNE requires real-valued input) and generated a 2D scatter plot. The visualization shows several distinct clusters, which suggests the model has learned to group semantically similar entities together. We tried to color the entities by their ontology class as requested, but ran into a limitation. The Wikidata entities in our exported NT file don't consistently carry rdf:type annotations. Without class labels, it is impossible to color the plot systematically. This is a limitation of working with raw Wikidata exports. We colored the known MJ-related entities (MJ himself in red, his nearest neighbors in blue) to at least show some semantic structure.

Relation Behavior

We used RotatE's phase analysis to study how different relation types are represented. In RotatE, a relation phase close to π indicates a symmetric relation, while values close to 0 indicate non-symmetric relations. All five relations we tested (P737, P463, P136, P264, P175) showed phases close to 0, correctly identifying them as non-symmetric directional relations. This matches our domain knowledge (we have musical influence directional, not symmetric). The inverse relation test (checking whether $||P264 * P175 - 1|| \approx 0$) gave values around 13.77, which is far from 0. This means the model didn't learn perfect inverse relationships between record label and performer predicates. This is likely due to insufficient training epochs rather than a model limitation.

5. RAG over RDF/SPARQL

The final part of the project was to build a chatbot that can answer natural language questions about Michael Jackson by querying our local Knowledge Base. The key challenge here was bridging the gap between natural language and SPARQL, using a small language model running entirely on our local machine.

5.1 System Setup

Component	Configuration
Machine	Windows laptop, CPU only, Anaconda Python 3.11
LLM	Gemma 2B served locally via Ollama (http://localhost:11434)
Knowledge graph	expanded_kb.nt loaded with rdflib 7.6.0 (66,719 triples)
API timeout	120 seconds per LLM call (increased from default due to slow CPU inference)

5.2 How the Pipeline Works

The RAG pipeline works in the following way. First, we load the expanded KB into a rdflib Graph object and build a schema summary. We get a compact representation of the graph structure with prefixes, a sample of distinct predicates, a sample of distinct classes and 20 example triples. This schema summary is important because we cannot send all 66k triples to the LLM. it needs to understand the graph structure without seeing the full data.

When the user asks a question, we send the SPARQL_INSTRUCTIONS prompt together with the schema summary and the question to Gemma 2B. The model generates a SPARQL query, which we extract from the response. After we inject Wikidata prefixes into and execute on the local graph with rdflib. If the execution raises a ParseException, the self-repair loop kicks in, we send the error message and the bad query back to the LLM and ask it to produce a corrected version.

5.3 Prompt Engineering Challenges

Getting a 2B parameter model to generate correct SPARQL was harder than we expected. The initial prompt from the lab template produced wrong results consistently: the model generated syntactically invalid queries (incomplete triples like '?person wd:Q2831 .' with no predicate), used wrong predicates (P97 instead of P737 for 'influenced by'), and sometimes added text after the closing code block which broke the query parser.

We solved these issues progressively. First, we added explicit property mappings to the prompt (P136 for genre, P264 for label, P166 for award, P737 for influenced by...) so the model doesn't have to guess. Second, we added a concrete example of a correctly structured triple with wd:Q2831 as the subject. Third, we modified the extract_sparql() function to cut everything after the last closing brace, removing any text artifacts the model might add after the query.

We also modified run_sparql() to inject Wikidata prefixes before executing any query. This was necessary because the generated queries always use wd: and wdt: prefixes but don't declare them.

5.4 Evaluation Results

Question	Baseline (LLM only)	RAG Result	Correct?
What is the music genre of Michael Jackson?	Pop — generic, no source	14 genre URIs from KB (P136)	✓ Yes
Which record label did MJ work with?	Epic Records — generic, partially correct	7 label URIs from KB (P264)	✓ Yes
Who influenced Michael Jackson?	James Brown, etc. — hallucinated list	0 results (LLM used P97 instead of P737)	✗ No
What awards did MJ receive?	13 Grammys — generic, incomplete	22 award URIs from KB (P166)	✓ Yes
Which artists have the same music style?	Justin Timberlake, etc. — hallucinated	0 results (P138 not in KB)	✗ No

We achieved 3/5 correct answers (60%). The three successful questions all follow the same simple pattern: SELECT ?x WHERE { wd:Q2831 wdt:Pxxx ?x }, which Gemma 2B handles well once we provide the right property mapping in the prompt.

The two failures are instructive. For question 3, the model chose P97 (noble title) instead of P737 (influenced by), showing that even with explicit mappings, the model can get confused by similarly numbered properties. Adding P737 explicitly to the prompt would fix this. For question 5, the concept of 'same music style' cannot be expressed as a simple triple pattern. it would require a JOIN query finding artists who share the same genre as MJ, which is beyond what Gemma 2B can generate reliably.

A key advantage of RAG over the baseline is grounding. The baseline gives generic, unverifiable answers, while our RAG system provides actual Wikidata URIs that can be traced back to the source. Even when the answers are just URIs rather than human-readable labels, they represent real facts from a KB rather than hallucinated content.

5.5 Self-Repair Mechanism

After our prompt improvements, the self-repair loop was rarely needed (Repaired=False). This means the enhanced prompt was sufficient to generate syntactically valid SPARQL in the first attempt. The remaining failures were semantic errors, which the self-repair loop cannot fix since it only corrects syntax errors.

5.6 Scalability of this Approach

Our current implementation uses rdflib as an in-memory triple store, which works for 66k triples but would not scale to millions of triples. For larger KGs, we would need to deploy a dedicated SPARQL endpoint. There are some limit like if we have hundreds of predicates and millions of entities, the summary would exceed the LLM context window. A more scalable approach would use vector similarity to retrieve only the most relevant schema elements for each question. Finally, Gemma 2B's limited capacity means we need very detailed prompt engineering. A larger model or a model fine-tuned specifically on SPARQL generation would improve accuracy.

```
=== EXEMPLE 1 ===
Question : What music genres are associated with Michael Jackson (Q2831)?

--- Baseline ---
Michael Jackson's music spans multiple genres, but he is primarily associated with the following styles:

1. Pop: Michael Jackson is widely regarded as one of the most successful pop artists of all time, with hits like "Billie Jean," "Beat It," and "Thriller."
2. R&B (Rhythm and Blues): As a member of The Jackson 5, Michael Jackson's music was heavily influenced by soul and R&B, with songs like "I

--- RAG ---

Question : What music genres are associated with Michael Jackson (Q2831)?
[Repaired] False
[SPARQL]
SELECT ?genre WHERE { wd:Q2831 wdt:P136 ?genre . }

[Résultats] (14 lignes)
genre
http://www.wikidata.org/entity/Q11401
http://www.wikidata.org/entity/Q45981
http://www.wikidata.org/entity/Q58339
http://www.wikidata.org/entity/Q840065
http://www.wikidata.org/entity/Q2529400
http://www.wikidata.org/entity/Q11403
http://www.wikidata.org/entity/Q37073
http://www.wikidata.org/entity/Q9759
http://www.wikidata.org/entity/Q211756
http://www.wikidata.org/entity/Q164444
http://www.wikidata.org/entity/Q131272
http://www.wikidata.org/entity/Q11399
http://www.wikidata.org/entity/Q316930
http://www.wikidata.org/entity/Q83270
```

```
=== EXEMPLE 2 ===
Question : Which awards did Michael Jackson (Q2831) receive according to the knowledge base?

--- Baseline ---
I'm not able to access the specific information about Michael Jackson's awards in my current knowledge cutoff. However, I can suggest some possible sources where you may be able to
find this information.

Michael Jackson was a renowned musician and artist who received numerous awards and accolades throughout his career. He won 13 Grammy Awards, among many others. If you're looking fo
r more specifi

--- RAG ---

Question : Which awards did Michael Jackson (Q2831) receive according to the knowledge base?
[Repaired] False
[SPARQL]
SELECT ?award WHERE { wd:Q2831 wdt:P166 ?award . }

[Résultats] (22 lignes)
award
http://www.wikidata.org/entity/Q1459443
http://www.wikidata.org/entity/Q953746
http://www.wikidata.org/entity/Q1027904
http://www.wikidata.org/entity/Q19858142
http://www.wikidata.org/entity/Q135498
http://www.wikidata.org/entity/Q1333798
http://www.wikidata.org/entity/Q90415505
http://www.wikidata.org/entity/Q4743376
http://www.wikidata.org/entity/Q16665736
http://www.wikidata.org/entity/Q17985761
http://www.wikidata.org/entity/Q61669648
http://www.wikidata.org/entity/Q935843
http://www.wikidata.org/entity/Q4992254
http://www.wikidata.org/entity/Q947090
http://www.wikidata.org/entity/Q843219
http://www.wikidata.org/entity/Q179191
http://www.wikidata.org/entity/Q15692068
http://www.wikidata.org/entity/Q1542347
http://www.wikidata.org/entity/Q904528
http://www.wikidata.org/entity/Q2976730
... (affichage limité à 20 sur 22)
```

6. Critical Reflection

Looking back at the full pipeline, there are several design choices and limitation

6.1 The French NER Decision and Its Consequences

Using fr_core_news_lg instead of en_core_web_trf was a pragmatic choice given our French sources, but it had cascading effects throughout the pipeline. French-form entities like 'Epic_Records30' or 'Biographie_courte' could not be matched in Wikidata, leading to failed alignments. These unaligned entities entered the KB with private URIs and no Wikidata connections, creating isolated subgraphs with limited connectivity. A better approach would have been to use a multilingual NER model or to post-process entity labels to normalize them before alignment.

6.2 Entity Count Exceeding the Recommendation

Our final KB contains 54,768 entities, roughly double the recommended maximum of 30,000. This happened because our 2-hop expansion was too broad: by following all predicates of MJ-related entities, we pulled in many peripheral nodes that have very little to do with Michael Jackson specifically. These extra entities add noise rather than signal to the embedding training. In hindsight,

we should have restricted the 2-hop expansion to a curated set of predicates (P136, P166, P264, P737) to keep the graph focused and dense. The impact is visible in the embedding results

6.3 Open-World vs Closed-World Assumption

RDF and OWL knowledge bases operate under the Open-World Assumption (OWA): if a fact is not present in the KB, it does not mean it is false it simply means we do not know. Knowledge Graph Embedding models, assume the Closed-World Assumption during training. The missing triples are used as negative examples. This mismatch is a tension in KGE. In our case, many facts that are true about Michael Jackson are simply absent from our KB export (for example we verified that wdt:P570, his date of death, was not in our graph). During training, the model treats the absence of these facts as evidence they are false, which introduces incorrect negative signals and degrades performance.

6.4 Impact of Predicate Alignment on Embeddings

The quality of our predicate alignment affected what the embeddings could learn. Well-aligned predicates (P166, P264, P737) created structural patterns in the graph that the models could learn from. But our generic predicate prop:isRelatedTo, which was too broad to map to any Wikidata property, added noise: many entities are connected by this relation, which dilutes the semantic signal. Better ontology design with specific predicates for each relation type would have produced cleaner embeddings.

6.5 What We Would Do Differently

- Use a multilingual NER model to improve entity extraction and Wikidata alignment rates
- Apply stricter expansion filters to keep entity count below 30,000 for denser graph connectivity
- Train KGE models with GPU acceleration and at least 500 epochs for proper convergence
- Use a larger LLM for more reliable SPARQL generation
- Add rdfs:label triples so that results display human-readable names instead of Q-ids
- Include rdf:type annotations in the KB expansion to enable proper t-SNE class coloring
- Build a web interface for the RAG chatbot to demonstrate it more clearly

Conclusion

This project gave us a hands-on understanding of the full Knowledge Graph engineering pipeline, from raw web data all the way to a working RAG chatbot. Each step taught us something concrete: NER ambiguity is harder to handle than it looks, Wikidata alignment depends heavily on entity label quality, graph size has a non-linear effect on embedding performance, and getting a small LLM to generate SPARQL requires careful prompt.

Looking at our results overall, we are satisfied with the scope of what we built but critical of some implementation choices. The RotatE model achieving MRR=0.196 on a noisy, oversized graph is a reasonable result given the constraints (CPU only, 100 epochs). The RAG chatbot achieving 60% accuracy on simple factual questions demonstrates that the approach works, even if scaling it to complex questions would require a more powerful model.

Lab	Main Deliverable	Key Result
Lab 1 — Web Mining	crawler_output.jsonl + entities_mJackson.csv	2,005 entities from 10 sources

Lab	Main Deliverable	Key Result
Lab 2 — KB Construction	expanded_kb.nt + ontology_alignment.ttl + entity_alignment.xls	66,719 triples, 98 relations
Lab 3 — Reasoning & KGE	train/valid/test.txt + 4 trained models	RotatE best: MRR=0.196, Hits@10=0.301
Lab 4 — RAG Chatbot	lab_rag_sparql_gen.py + README.md	60% accuracy, grounded KB answers

All code, data files, and trained models are available in the GitHub repository. The README.md provides step-by-step instructions for reproducing every result on any machine with Python 3.9+ and Ollama installed.